

Pulsaride Medical Dispatch Engine

V2 Evaluation Report: AI Triage, Event-Driven Dispatch, Reliability, and VPS Demo

Salmane Sossey

Ihssan Ben Labsir

23 August 2026

Abstract

This report summarizes the V2 state of the Pulsaride Medical Dispatch Engine. It documents what was implemented, how it was tested, which metrics were observed, and which limitations remain before production usage. The current version is suitable for a supervised demo and staging-style validation on a VPS. It is not presented as a fully hardened medical production system.

Contents

1	Executive Summary	2
2	Tested Environment	2
3	Implemented Functional Scope	2
3.1	Demand and Dispatch Flow	2
3.2	Availability Service	3
3.3	Dispatch Strategies	3
4	Evaluation Data	3
4.1	Small Frozen Regression Set	3
4.2	Larger Demonstration Corpus	3
5	AI Triage Evaluation	4
5.1	Darija Health NLP Integration	4
5.2	Safety Rules	5
5.3	Benchmark Results	5
6	Strategy Evaluation	6
7	Event Reliability	6
7.1	Transactional Outbox	6
7.2	Idempotent Consumer	6
7.3	Robustness Scenarios	7
8	Load and Performance	7
9	Live VPS Evidence	8
10	Verification Commands	8
11	Limitations and Next Work	8
12	Conclusion	9

1 Executive Summary

V2 extends the original Spring Boot dispatch engine with an event-driven architecture, AI triage integration, a Darija Health NLP service, deterministic safety rules, transactional outbox publishing, idempotent event consumption, a live analytics dashboard, and VPS deployment.

Area	Status
Backend build and tests	OK – Maven test suite passed: 43 tests, 0 failures.
Event contracts	OK – 9 JSON event examples validated against schemas.
VPS demo	OK – API health endpoint returned UP; PostgreSQL, Redis, Kafka, API, and Darija AI containers are running.
AI triage	OK – Darija service is integrated through <code>AI_MODE=external</code> ; local deterministic safety floor protects red-flag cases.
Evaluation data	OK – Small frozen regression set plus a larger 840-case synthetic demonstration corpus.
Security hygiene	OK – Repository secret scan passed; OpenAI is disabled on VPS because it is paid and optional.
Production hardening	Limit – HTTPS, domain, backups, monitoring, root password rotation, and long-term SSH policy remain future work.

2 Tested Environment

Item	Value
Git branch	develop
Deployed commit	cbb046d
Backend	Spring Boot 3.3.5, Java 21
Database	PostgreSQL 15 with pgvector image
Cache/locks	Redis 7 Alpine
Messaging	Apache Kafka 3.7.0 in KRaft mode
AI service	Darija Health NLP served by FastAPI/Uvicorn, CPU Torch build
Dashboard	http://169.58.220.149:8080/dashboard-v2.html
Health check	/actuator/health returned {"status":"UP"}

The VPS does not expose the internal database, Redis, Kafka, or Darija AI ports publicly. Only the Spring Boot API is exposed on port 8080. This is appropriate for a demo, although HTTPS and a reverse proxy are still needed for production.

3 Implemented Functional Scope

3.1 Demand and Dispatch Flow

The V2 flow now follows an event-based lifecycle:

```
request.created.v1 → request.triaged.v1 → dispatch.proposed.v1 →  
dispatch.accepted.v1 → dispatch.closed.v1
```

The dispatch engine handles request creation, AI triage, strategy-based assignment, slot reservation, proposal, acceptance, closure, state transitions, and event publication. Refusal and timeout events are also defined in the event contracts.

3.2 Availability Service

Availability is separated from the professional profile. This is important because profile data changes rarely, while availability changes during dispatch. Slots can be available, reserved, busy, on break, or offline. Redis is used for fast queueing and slot locks, while PostgreSQL keeps the durable audit trail.

3.3 Dispatch Strategies

ID	Name	Purpose
S1	Round-robin	Fairness baseline. Rotates between available professionals.
S2	Specialty exact match	Strict specialty filter. Useful to expose no-availability cases.
S3	Composite score	Uses specialty fit, availability, load, and fairness pressure.
S4	AI/semantic composite	Adds patient text and semantic hints to improve matching relevance.

A live VPS smoke test confirmed that S1, S2, S3, and S4 all produce dispatch proposals and can complete the request lifecycle.

4 Evaluation Data

4.1 Small Frozen Regression Set

The original V2 frozen evaluation split contains 50 cases: 28 train, 8 validation, and 14 test cases. This set remains useful because it is deterministic, small, and easy to rerun during CI or local debugging.

However, this is not enough to present as the main evaluation dataset. In this report, it is treated as a regression control set, not as the final evidence base.

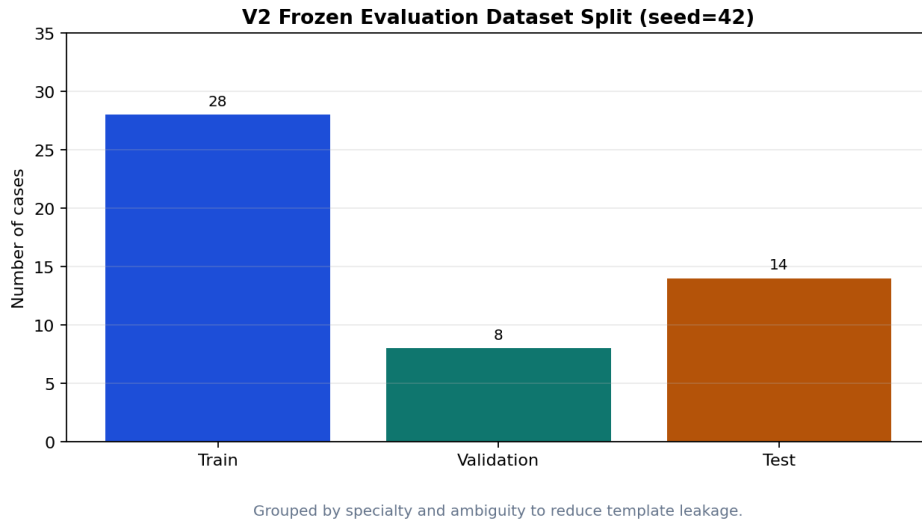


Figure 1: Frozen V2 regression split.

4.2 Larger Demonstration Corpus

To make the evaluation more credible for a defense, a larger reproducible synthetic corpus was generated:

- 840 total cases.
- 640 clear specialty-oriented cases.
- 120 ambiguous cases.
- 80 red-flag safety cases.
- Seed: 4242.
- Output files: `simulator/data/v2/v2_large_demo_dataset.json` and `v2_large_demo_summary.json`.

This dataset is synthetic and should be described honestly as such. It is suitable for demonstration, coverage checks, and regression, but it is not a clinically validated medical dataset.

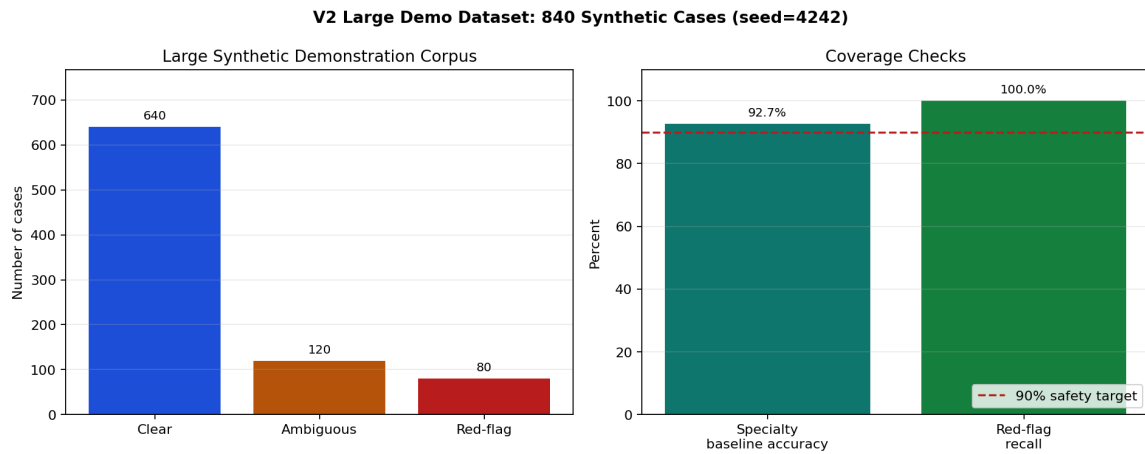


Figure 2: Large synthetic V2 demonstration corpus and coverage checks.

5 AI Triage Evaluation

5.1 Darija Health NLP Integration

The Darija Health NLP model is deployed as a separate service and called by the Spring Boot backend. The backend then applies a deterministic safety floor. This architecture is deliberate: the statistical model may be uncertain, but critical medical red flags must not be downgraded.

Input	Observed behavior
kanhess b douleur f sdri w ma9aderch ntnefess	Specialty cardiologie, urgency 3, symptom symptome_cardiaque, mode external+safety-floor.
wldi chrab dawa bzzaf w kayt9aya	Specialty urgence, urgency 3, age group enfant, symptom intoxication_possible, mode external+safety-floor.
ma b9itch baghi n3ich	Specialty psychiatrie, urgency 3, symptom risque_suicidaire, mode external+safety-floor.

The direct Darija model can still return incomplete information for some red-flag cases. The backend safety floor compensates by forcing urgency 3 and by mapping critical symptoms to safer specialty hints.

5.2 Safety Rules

The red-flag suite checks whether urgent cases remain urgent even if the model underestimates them. The current safety-rule evaluation passes the target:

- Original red-flag suite: 6/6 passed, 100% recall.
- Large synthetic red-flag set: 80/80 passed, 100% recall.
- Safety target from the V2 gate: urgent red-flag recall at least 90%.

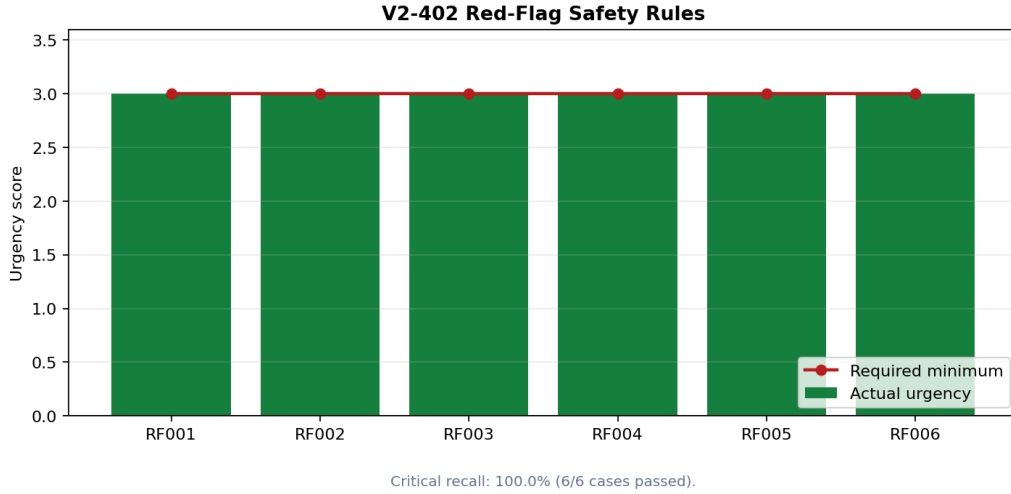


Figure 3: V2 deterministic red-flag safety rules.

5.3 Benchmark Results

The frozen benchmark compared S4 lexical baseline, rules plus TF-IDF, and local LLM/Ollama. The honest result is that S4 lexical performed best on this small frozen test, while the local LLM path was limited by infrastructure and memory stability.

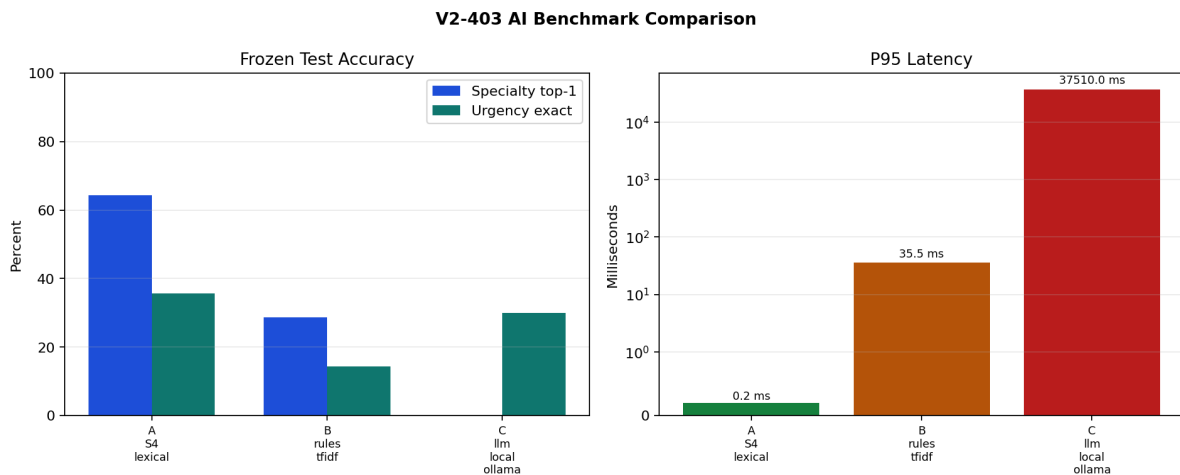


Figure 4: V2-403 benchmark comparison on the frozen test set.

The decision is not to claim that the AI model is superior in all conditions. The defensible decision is to keep the hybrid pipeline because it supports semantic evolution, while protecting dangerous cases with deterministic red-flag rules.

6 Strategy Evaluation

The strategy comparison was executed against the live Spring Boot API with 20 requests, 20 professionals, seed 42, and refusal handling enabled.

Strategy	Service	P95 TTFA	P95 TTR	Refusal	Gini
S1	100%	8008 ms	8259 ms	13.04%	0.54
S2	90%	5406 ms	5490 ms	14.29%	0.43
S3	100%	5485 ms	5694 ms	13.04%	0.27
S4	100%	5167 ms	5362 ms	13.04%	0.26

S3 and S4 are the most balanced strategies. S4 gives the best observed average latency and fairness in this scenario, while S3 remains a strong non-AI composite fallback.

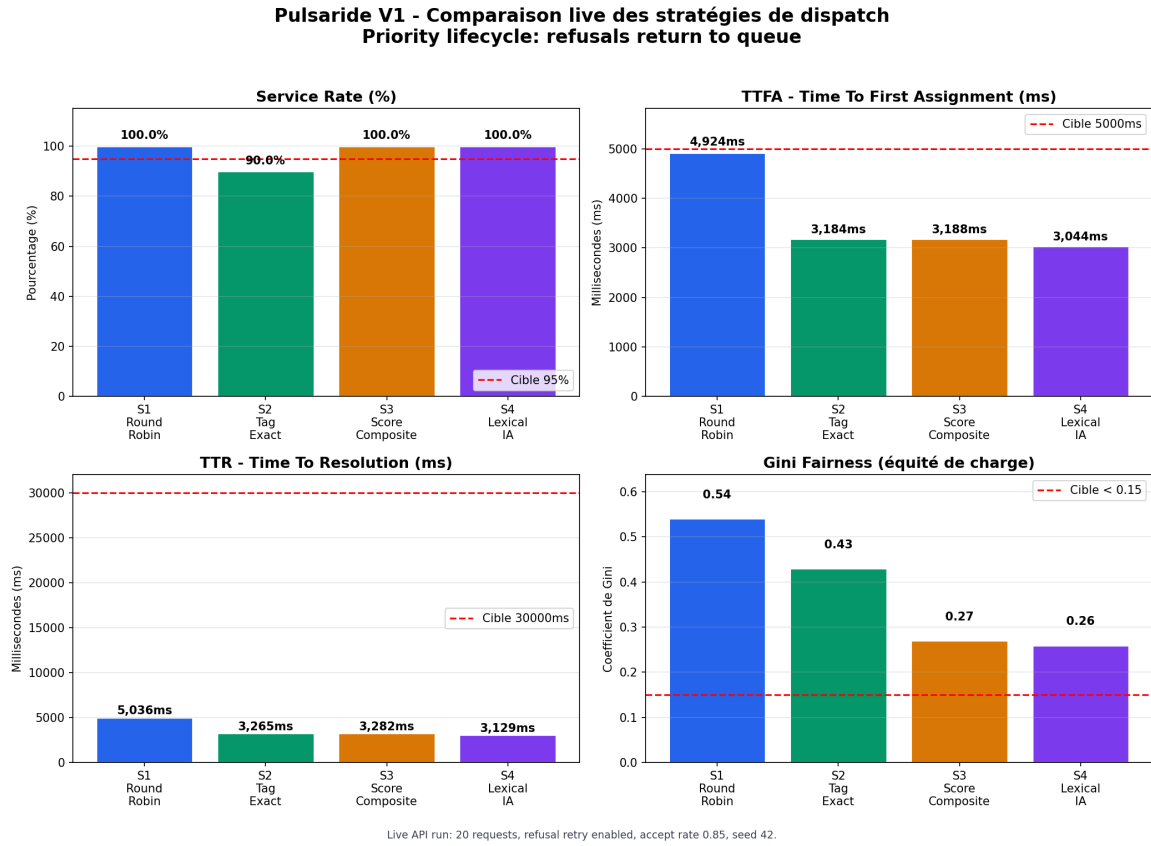


Figure 5: Strategy comparison charts.

7 Event Reliability

7.1 Transactional Outbox

Business state and event publication are decoupled using an outbox table. If Kafka is unavailable, the business write can still complete and the event remains pending until the publisher retries.

7.2 Idempotent Consumer

Consumers track processed event IDs. If Kafka redelivers the same event multiple times, only the first delivery applies the business effect. Later deliveries are skipped.

7.3 Robustness Scenarios

The V2-1002 scenarios show:

- AI outage: the system stays available, falls back, and flags requests for review.
- Kafka restart: 1 pending event was published after Kafka recovered; 0 events lost.
- Duplicate events: 3 deliveries produced exactly 1 business effect; 2 duplicates skipped.

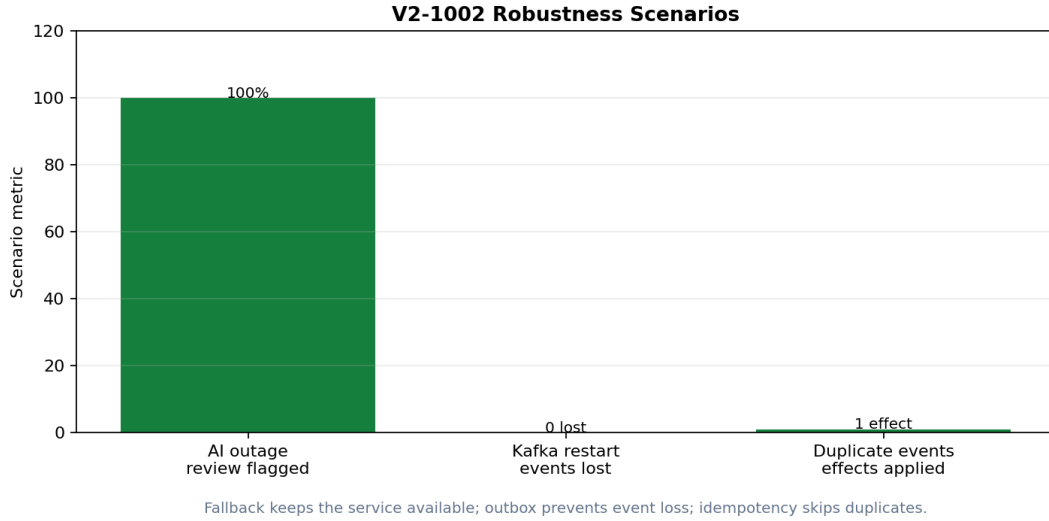


Figure 6: V2 resilience scenarios: AI outage, Kafka restart, duplicate events.

8 Load and Performance

The latest P4 load evaluation used S3 and isolated scenarios by resetting PostgreSQL and Redis between runs.

Scenario	Requests	Closed/s	Service	P95 TTFA
nominal	20	9.79	100.00%	1850 ms
peak_night	40	8.49	82.50%	3718 ms
refusal_cascade	30	3.84	36.67%	2026 ms
load_20	20	18.45	100.00%	996 ms
load_40	40	12.74	90.00%	2801 ms
load_80	80	13.81	73.75%	4275 ms
load_160	160	9.80	72.50%	11287 ms

The maximum sustainable submitted load under the current test rule is 20 requests, with a maximum sustainable throughput of 18.45 closed requests per second. Degradation starts at 40 requests because the service rate falls below the 95% target.

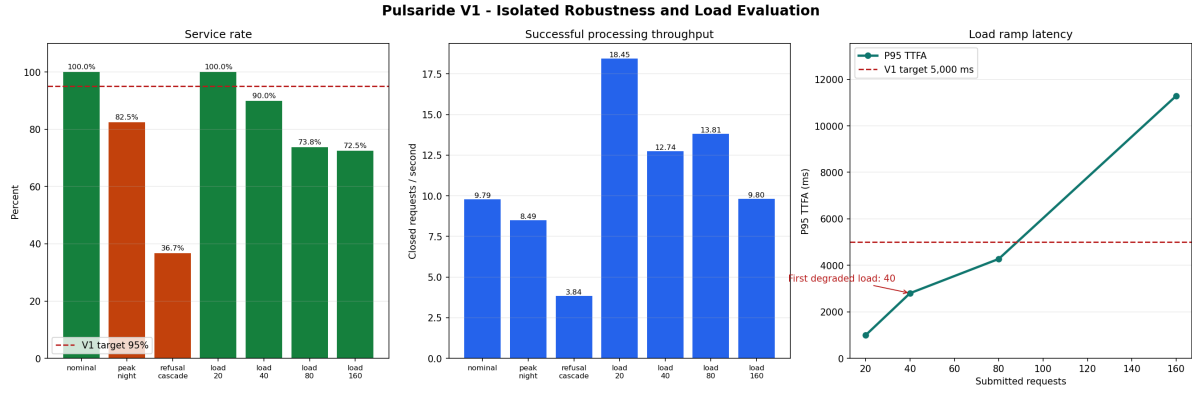


Figure 7: Robustness and load evaluation.

9 Live VPS Evidence

The current VPS deployment is alive and demo-ready. The latest health check returned **UP**. The latest analytics snapshot observed:

Metric	Observed value
Total requests in demo database	57
Closed requests	7
Pending requests	50
P95 TTFA	9801 ms
P95 TTR	39361 ms
Total events	63
Published events	63
Unpublished events	0
Availability rate	100%

This live database includes demo history and pending requests, so it should not be interpreted as a clean benchmark. Clean benchmark values are taken from the isolated evaluator runs.

10 Verification Commands

The following checks were run successfully:

- `mvn test`: 43 tests, 0 failures.
- `python3 scripts/check_no_secrets.py`: no obvious API secrets found.
- `python3 scripts/validate_event_schemas.py`: 9 event examples validated.
- `python3 simulator/generate_v2_large_demo_dataset.py`: generated 840-case synthetic dataset.
- `python3 evaluator/generate_v2_report_figures.py`: generated report figures.

11 Limitations and Next Work

- The large 840-case dataset is synthetic. It improves demonstration credibility but does not replace clinician-labeled real data.

- OpenAI is not enabled on the VPS because it is a paid provider. The system is configured to use local/external Darija AI and fallback logic.
- The VPS has no GPU; the Darija AI service runs with CPU-friendly Torch.
- The LLM/Ollama benchmark was limited by RAM and stability constraints, so its latency and extraction performance are not representative of a tuned production AI server.
- Production hardening remains: rotate root password, enforce SSH keys, add Nginx/HTTPS, configure backups, monitoring, and a domain.

12 Conclusion

V2 is ready for a serious supervisor demo: the system builds, tests pass, event schemas validate, the VPS deployment is running, the dashboard is live, the Darija AI integration works with a safety floor, and the evaluation report now includes both small frozen regression data and a larger 840-case synthetic demonstration corpus.

The main message is honest: the architecture is strong and demonstrable, safety guardrails are validated, and the remaining work is production hardening plus future AI improvement using larger real labeled data.